Universidad de San Carlos de Guatemala

Centro Universitario de Occidente

División de Ciencias de la Ingeniería

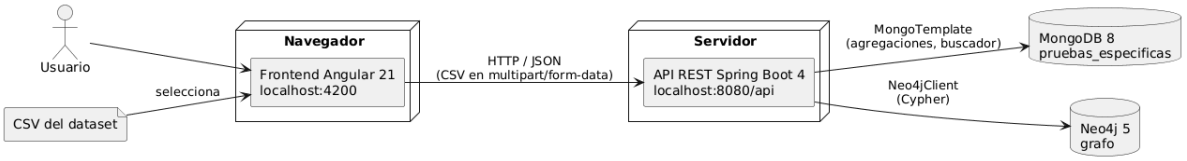
Sistemas de bases de datos 2

Ing. Daniel Gonzales

Manual Técnico - Proyecto Final

Angel Abelino Son Vicente 202030885

Arquitectura



- Frontend: pantallas de carga, dashboard, explorador de grafos y buscador. Solo habla con la API.
- Backend: recibe los CSV, los limpia (ETL) y guarda cada evaluación en las dos bases. También ejecuta las consultas y devuelve JSON.
- MongoDB: guarda los documentos y responde las 20 consultas de agregación y el buscador.
- Neo4j: guarda las mismas evaluaciones como un grafo y responde las 5 consultas Cypher del explorador.

Los pipelines de MongoDB y las consultas Cypher no están escritos en Java: son archivos en backend/src/main/resources/queries/ que el backend lee al iniciar. Así se pueden copiar tal cual a mongosh o a Neo4j Browser, y se pueden modificar sin tocar el código.

Tecnologías

- Frontend: Angular 21 (componentes standalone y signals), Angular Material, Apache ECharts con ngx-echarts.
- Backend: Java 21, Spring Boot 4.1 (Web MVC, Validation, Data MongoDB, Data Neo4j), Apache Commons CSV.
- Bases de datos: MongoDB 8.0 y Neo4j 5.26 Community, levantadas con Docker Compose.
- Pruebas: JUnit 5 y MockMvc en el backend; Vitest en el frontend.

Instalación y ejecución

1. Levantar las bases de datos: docker compose up -d.
2. Backend: cd backend && ./mvnw spring-boot:run (puerto 8080).
3. Frontend: cd frontend && npm ci && npm start (puerto 4200).
4. Cargar los CSV de data/ desde la pantalla de carga.

Para generar los artefactos de producción:

cd backend && ./mvnw package # target/backend-0.0.1-SNAPSHOT.jar

cd frontend && npx ng build # dist/frontend

Variables de entorno

Todas tienen un valor por defecto que coincide con docker-compose.yml, así que en desarrollo no hace falta definir ninguna.

- MONGO_USER y MONGO_PASSWORD (Docker Compose): admin / admin123.
- NEO4J_USER y NEO4J_PASSWORD (Docker Compose y backend): neo4j / neo4j12345.
- MONGO_URI (backend): mongodb://admin:admin123@localhost:27017/pruebas_especificas?authSource=admin.
- NEO4J_URI (backend): bolt://localhost:7687.
- SERVER_PORT (backend): 8080.
- CORS_ORIGINS (backend): http://localhost:4200.

La URL de la API que usa el frontend está en frontend/src/environments/environment.ts.

Proceso de carga (ETL)

El archivo se lee en streaming, fila por fila, sin cargarlo completo en memoria. Las filas válidas se acumulan en lotes de 1000. Cada lote se escribe en MongoDB con bulkWrite y en Neo4j con UNWIND + MERGE.

Lectura del archivo

- Delimitador: se detecta si es coma o punto y coma.
- Encabezados: se aceptan con mayúsculas, espacios o BOM. Las columnas pueden venir en cualquier orden.
- Encabezados cortados: los archivos 2022-2024 traen nombres de columna recortados (numero_de_fecha_de_evaluaci, municipio_institucion_). Se aceptan si son el inicio de una sola columna esperada y tienen al menos 10 caracteres.
- Columnas faltantes: si falta una columna obligatoria, el archivo completo se marca como fallido con la lista de columnas que faltan.
- Archivos que no son CSV: se marcan como fallidos ("Solo se aceptan archivos .csv"), sin afectar a los demás archivos del mismo envío.

Validación: causas de rechazo

Una fila se rechaza, y se guarda con todas sus causas, si:

- el correlativo o la materia vienen vacíos;
- la fecha de asignación no se puede interpretar (se aceptan aaaa-mm-dd, d/m/aaaa, d-m-aaaa y aaaa/m/d);
- el número de evaluación no es un entero mayor que 0;
- el año de ingreso no es un entero entre 2000 y 2100;
- el resultado no es APROBADO o NO APROBADO (también se aceptan REPROBADO, SI y NO);
- la fila tiene una cantidad de columnas distinta a la del encabezado.

Reglas de limpieza

- Textos: sin espacios sobrantes, en mayúsculas y sin tildes (la Ñ se conserva), para que "Público" y "PUBLICO" sean el mismo valor.
- Vacíos: "", SIN REGISTRO, SIN DATOS, NULL, N/A, NA y - se guardan como SIN REGISTRO.
- Sexo: M o F; cualquier otro valor queda como SIN REGISTRO.
- Carrera: NINGUNO o vacío se guarda como SIN CARRERA.
- Año de nacimiento: si no es un año válido o es posterior al ingreso, queda como dato faltante (no se rechaza la fila).
- Edad: año de ingreso menos año de nacimiento; se descarta si queda fuera de 14 a 70 años.
- Correlativo: en minúsculas y con sus primeros 20 caracteres (ver abajo).
- Carrera y municipio cortados: se completan con los catálogos (ver abajo).

Particularidades de los archivos 2022-2024

Los archivos de 2015 a 2021 traen valores completos. Los de 2022 en adelante traen las columnas en otro orden y varios textos cortados. El ETL los resuelve así:

- Correlativo:
    - Problema: antes de 2022 tiene 32 caracteres (un hash); desde 2022 trae solo los primeros 20 a 23 del mismo hash.
    - Solución: se guardan los primeros 20 caracteres en todos los años. En el dataset no hay dos aspirantes que compartan esos 20 caracteres, y así se enlazan los 2,174 aspirantes que aparecen en los dos periodos. Las consultas y el buscador aplican el mismo recorte al correlativo que reciben, así que también se puede buscar con los 32 caracteres.
- Carrera y municipio:
    - Problema: vienen cortados, por ejemplo INGENIERIA EN CIENCIAS Y o ANTIGUA GUATEMAL.
    - Solución: se comparan con los catálogos resources/catalogos/carreras.txt y municipios.csv, armados con los valores completos de 2015-2021, y se aplican estas reglas:
        - Si el texto coincide con un solo valor del catálogo, se usa ese valor.
        - Un espacio al final indica que la palabra siguiente se perdió: INGENIERIA MECANICA no es lo mismo que INGENIERIA MECANICA.
        - Si coincide con dos o tres valores, se guarda de forma explícita: INGENIERIA MECANICA (ELECTRICA O INDUSTRIAL).
        - Si coincide con más de tres, se guarda como GUATEMALA - CIUDAD CAPITAL ZONA (SIN ESPECIFICAR), porque la zona se perdió.
        - Si no coincide con ninguno, se guarda tal cual (puede ser un municipio que no aparece antes de 2022).
- Filas repetidas: algunos archivos traen filas idénticas repetidas (834 en total). Cada evaluación tiene un identificador determinístico, correlativo|materia|fecha|intento|año de ingreso, así que la repetida reemplaza a la anterior y se cuenta como duplicado.

Modelo de datos en MongoDB

Base de datos pruebas_especificas.

evaluaciones

Un documento por evaluación (fila válida). Es la colección de la que leen casi todas las consultas.


```
{

  "_id": "949d241ac10256edf113|MATEMATICA|2014-08-23|1|2015",

  "correlativo_aspirante": "949d241ac10256edf113",

  "fecha_asignacion": ISODate("2014-08-23T00:00:00Z"),

  "anio_asignacion": 2014,

  "mes_asignacion": 8,

  "sexo": "M",

  "anio_nacimiento": null,

  "edad": null,

  "materia": "MATEMATICA",

  "intento": 1,

  "anio_ingreso": 2015,

  "aprobado": false,

  "carrera_objetivo": "SIN CARRERA",

  "departamento": "GUATEMALA",

  "municipio": "GUATEMALA - CIUDAD CAPITAL ZONA 1",

  "tipo_institucion": "PRIVADO",

  "carga_id": ObjectId("…"),

  "archivo": "pruebas_especificas_2015.csv"

}
```

Algunos campos se guardan ya calculados (edad, anio_asignacion, mes_asignacion) para no recalcularlos en cada consulta.

Índices:

- correlativo_aspirante
- {materia, aprobado}
- {carrera_objetivo, anio_ingreso}
- tipo_institucion
- {departamento, municipio}

El _id empieza con el correlativo, así que ordenar por _id agrupa las evaluaciones de cada aspirante.

aspirantes

Un documento por aspirante, con sus evaluaciones embebidas. Sirve para consultar el historial completo de un aspirante con una sola lectura y para contar aspirantes sin agrupar las evaluaciones .

{

"_id": "949d241ac10256edf113",

"sexo": "M",

"carrera_objetivo": "SIN CARRERA",

"carreras": ["SIN CARRERA"],

"materias": ["MATEMATICA", "COMPUTACION"],

"departamento": "GUATEMALA",

"municipio": "GUATEMALA - CIUDAD CAPITAL ZONA 1",

"tipo_institucion": "PRIVADO",

"evaluaciones": [

{ "evaluacion_id": "…", "materia": "MATEMATICA", "fecha_asignacion": ISODate("…"), "intento": 1,

"anio_ingreso": 2015, "aprobado": false, "edad": null, "carrera_objetivo": "SIN CARRERA", … }

]

}

Se actualiza con $set para los datos más recientes del aspirante y con $addToSet para las listas, así que una evaluación repetida no se agrega dos veces.

registros_rechazados

Una fila rechazada por documento:

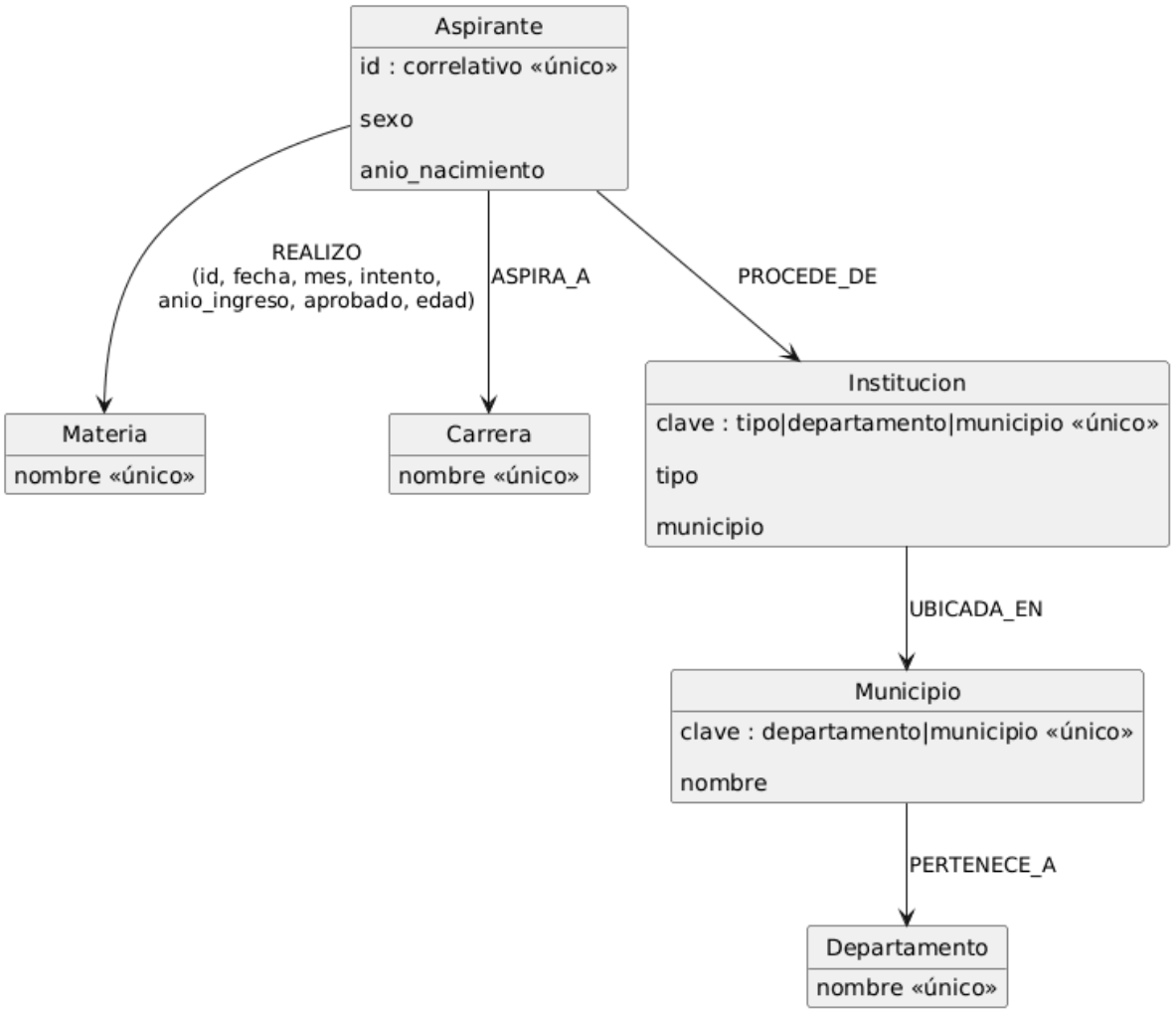
{ "carga_id": ObjectId("…"), "archivo": "…", "numero_fila": 5, "causas": ["…"], "causa": "…", "fila_original": { … } }

fila_original guarda los valores tal como venían en el CSV. Índice: {carga_id, numero_fila}.

cargas

Historial de cargas. Cada documento tiene archivo, estado (EN_PROCESO, COMPLETADO, COMPLETADO_CON_RECHAZOS o FALLIDO), las fechas de inicio y fin, los conteos (leidos, procesados, rechazados, duplicados), tiempo_ms y mensaje.

Modelo del grafo en Neo4j



- Instituciones: el dataset no trae su nombre. Por eso una institución se identifica por su tipo y su ubicación.
- Municipios: se identifican junto con su departamento, porque hay municipios con el mismo nombre en distintos departamentos.
- Evaluaciones: cada evaluación es una relación REALIZO con las propiedades id, fecha, mes, intento, anio_ingreso, aprobado y edad. Así, los intentos de un aspirante en una materia son varias relaciones entre los mismos dos nodos.
- Índices: cada clave única es una restricción CREATE CONSTRAINT … IS UNIQUE, que además crea el índice que usan los MERGE de la carga. Las relaciones REALIZO tienen un índice sobre id.

API REST

Todas las rutas empiezan con /api. Las respuestas son JSON.

- POST /cargas: recibe uno o varios CSV en el campo archivos (multipart) y devuelve el resumen de cada uno.
- GET /cargas: historial de cargas, de la más reciente a la más antigua.
- GET /cargas/{id}: detalle de una carga.
- GET /cargas/{id}/rechazados?pagina=&tamano=: filas rechazadas de una carga, paginadas.
- GET /mongo: lista de las 20 consultas con sus parámetros.
- GET /mongo/{id}?correlativo=: ejecuta una consulta; devuelve título, descripción, tiempo y filas.
- GET /grafo: lista de las 5 consultas Cypher con sus filtros y su texto.
- GET /grafo/{id}?filtro=valor&limite=: ejecuta una consulta Cypher; devuelve nodes, links y filas.
- GET /buscar: busca evaluaciones con filtros combinados y paginación (máximo 100 por página). Acepta aspirante, carrera, institucion, materia, municipio, departamento, anio, aprobado, pagina y tamano.
- GET /buscar/opciones: valores distintos de carrera, materia, departamento, municipio, tipo y año, para los filtros.
- GET /health: estado de MongoDB y Neo4j (200 si ambas responden, 503 si alguna falla).

Manejo de errores

Un @RestControllerAdvice convierte cualquier error en una respuesta con el mismo formato:

{ "estado": 400, "error": "El parámetro 'limite' debe estar entre 1 y 500", "detalle": null,

"ruta": "/api/grafo/2", "timestamp": "…" }

Los códigos son 400 para parámetros inválidos o faltantes y envíos sin archivos, 404 para una consulta o carga inexistente, 413 para un archivo de más de 200 MB, 503 cuando MongoDB o Neo4j no están disponibles (con un mensaje que indica cuál falló) y 500 para cualquier otro error, sin exponer detalles internos.

En el frontend, un HttpInterceptor muestra el mensaje en un aviso (snackbar). Cada pantalla tiene estados de carga, vacío y error con opción de reintentar, así que la aplicación no se cierra ni queda en blanco aunque una base de datos esté apagada.

Frontend

Cada ruta tiene su componente en pages/:

- /dashboard: indicadores generales y las 20 consultas en pestañas, cada una como gráfica o tabla.

- /carga: carga de varios CSV con progreso, resultado por archivo, historial y filas rechazadas.
- /grafo: explorador de grafos con ECharts (serie graph, layout de fuerzas).
- /buscador: búsqueda de evaluaciones con filtros y tabla paginada.

Detalles de implementación:

- Carga diferida: cada pantalla se carga al entrar en ella, para que el bundle inicial sea liviano.
- Gráficas: se arman en pages/dashboard/graficas.ts y consultas.ts. Los colores siguen un orden fijo para que cada categoría conserve su color.
- Explorador de grafos: el color de cada tipo de nodo es fijo, el tamaño del nodo depende de cuántas conexiones tiene y el grosor de la línea depende del valor de la relación.

Conclusiones

Sobre la solución

- Dos modelos para una misma fuente funcionan bien si cada uno responde lo que le corresponde.
    - Cada fila del CSV se guarda dos veces: como documento en MongoDB y como relación REALIZO en Neo4j.
    - MongoDB responde las estadísticas (conteos, promedios, porcentajes, series por año).
    - Neo4j responde las preguntas sobre conexiones (qué une a un aspirante con su departamento, qué aspirantes comparten municipio y carrera).
    - Ninguna de las dos bases tuvo que resolver lo que la otra hace mejor.
- Las dos bases se mantienen consistentes sin una transacción distribuida.
    - Cada evaluación tiene un identificador determinístico, correlativo|materia|fecha|intento|año.
    - MongoDB escribe con upsert sobre ese identificador y Neo4j con MERGE.
    - Así, cargar un archivo dos veces, o reintentar una carga interrumpida, deja exactamente los mismos datos.
    - Con los diez archivos, las dos bases quedaron con las mismas 207,619 evaluaciones.
- Validar con los datos reales fue de mucha ayuda.
    - Los archivos 2022-2024 tienen otro formato: columnas en otro orden, nombres de columna cortados, textos cortados y correlativos recortados.
    - Con datos de prueba el ETL funcionaba, pero habría rechazado esos tres archivos completos.
    - La solución combinó dos piezas:
        - reglas genéricas, como aceptar un encabezado que es el inicio de un solo nombre esperado;
        - catálogos tomados de los años con datos completos.
    - Cuando un valor no se puede recuperar se marca de forma explícita, en lugar de adivinarlo.

Ventajas de usar una base documental y una de grafos

MongoDB (documental)

- El pipeline de agregación cubre todas las consultas estadísticas con una sola herramienta: filtros, agrupaciones en varias etapas, porcentajes, $topN, fechas y

$out para una colección resumen. Las 20 consultas responden en menos de un segundo sobre 207 mil documentos.
- Los documentos guardan la fila casi tal cual, con campos calculados (edad, mes) que evitan cálculos repetidos.
- La colección aspirantes, con las evaluaciones embebidas, entrega el historial completo de una persona con una sola lectura por _id.
- Los pipelines se guardan como JSON, así que son fáciles de versionar, documentar y probar en mongosh.

Neo4j (grafos)

- Las consultas sobre relaciones son cortas y legibles. Recorrer Departamento, Municipio, Institución, Aspirante, Carrera es un solo MATCH, en MongoDB serían varios $lookup, o habría que duplicar todo en cada documento.
- La red de un aspirante (C1) sale de un recorrido de longitud variable, sin saber de antemano cuántos niveles tiene.
- Las restricciones de unicidad evitan nodos duplicados y a la vez crean los índices que hacen rápido el MERGE de la carga.
- El resultado se dibuja de forma natural como grafo, lo que ayuda a ver patrones (grupos de aspirantes, carreras dominantes por departamento) que en una tabla pasan desapercibidos.

Limitaciones

- Doble escritura y doble almacenamiento:
    - Cada evaluación se guarda dos veces, y la carga tarda más que con una sola base.
    - Si una base falla a media carga, la otra puede quedar adelantada. Se mitiga con la idempotencia (basta con volver a cargar), pero no hay una transacción que abarque las dos.
- Agregaciones en Neo4j:
    - Los conteos y porcentajes sobre todo el grafo son más lentos que en MongoDB.
    - Mostrar muchos nodos a la vez no es útil: con más de unos cientos el dibujo se vuelve ilegible. Por eso las consultas del explorador tienen un límite de 500.
- Relaciones en MongoDB:
    - Responder "quién está conectado con quién" exige $lookup o duplicar datos.
    - El documento del aspirante con evaluaciones embebidas crece con cada intento. Con este dataset no hay problema (el máximo es de unas decenas de evaluaciones), pero no escalaría a historiales muy largos.
- Datos de origen:
    - La calidad del dataset limita el análisis: 2015 no trae carrera objetivo, los archivos recientes pierden la zona de la capital y la distinción entre Mecánica Eléctrica e Industrial, y el año de nacimiento falta en buena parte de 2015.

    - Ninguna de las dos bases puede recuperar información que no viene en el archivo.
- Esquema flexible: MongoDB acepta cualquier documento, así que la consistencia de los campos depende del ETL y no de la base. Las reglas de limpieza y sus pruebas automáticas son las que garantizan que todos los documentos tengan la misma forma.